Hey, so I think we are nearly ready with Handy's Android app. It works fine to some extent and I think we can release it on Play Store as a V1 at least. Can you review the entire code and also what it takes to send an app to Play Store, where, what account do I need to create? Do I need to pay anything? How do I upload the app? How do I, you know, bundle an APK? I am literally unaware of the entire process it takes. So could you please do a very deep research, scan the web if required, come up with the best practices, read all the best practices from different developer forums, Android forums, and tell me step by step on what I need to do to get it out there. Let's not implement anything yet, but I want a very detailed plan on what I need to do to get it out there, how long does it typically take, the different touch points here, etc. and maybe even the money involved in Play Store fees or whatever. I'm not sure of how that works too.

This is going to need fresh research — Play Store policies, fees, and especially the AccessibilityService rules have shifted in 2025/2026. Let me pull the current docs and verify a few things in your code, then write a step-by-step plan.
Searched the web

Searched the web

Acknowledged — let me also pull current docs on direct APK sharing, then verify your build setup before writing the plan.
Searched 4 patterns, searched the web, read 2 files

Searched 4 patterns, searched the web, read 2 files

Got it — and yes, APK-via-Drive is the smart first move. The Play Store path has a mandatory 14-day closed-test wait for new personal accounts, so collecting real-world feedback via sideload while you stage the Play submission is the right sequence.
Here's the full plan, addressing both your questions.

Part 1 — What I checked in your code
Quick state check so the plan below is grounded:

- applicationId = "com.handy.android", versionCode = 1, versionName = "0.1.0". ✅ ready to bump per build.
- minSdk = 26, targetSdk = 36, compileSdk = 36. ✅ already at the Play Store's 2026 floor — most apps are scrambling to get to 35/36 this year, you're already there.
- Release build type has isMinifyEnabled = true and isShrinkResources = true with proguard-rules.pro. ✅
- PRIVACY_POLICY.md exists in the repo root. ✅ but needs to be **publicly hosted at a URL** before Play submission (and arguably before Drive distribution too, so testers can see it).

- **No signingConfig block in** app/build.gradle.kts**.** ❌ this is the one hard blocker. Release builds today either fail or produce unsigned APKs. We add a keystore step below.
- Manifest
  declares RECORD_AUDIO, SYSTEM_ALERT_WINDOW, POST_NOTIFICATIONS, FOREGROUND_SERVICE_* (SPECIAL_USE, MICROPHONE, MEDIA_PROJECTION), READ_CONTACTS, plus the AccessibilityService. **Every one of those is a sensitive permission that Play will review.**
- RuntimeModule has a debug-only trust-all SSL path. Already gated by isDebuggableApp() — release builds use the platform trust manager. ✅

Part 2 — Path A: ship as a signed APK via Google Drive (do this first)
This is the fastest path to real-user feedback. Total elapsed time once you sit down: ~2 hours.
Step 1 — Create a release keystore (one-time, do this once and back it up)
A keystore is a file containing a private signing key. The same key must sign every future update of your app on Play (or every future APK if you stay on sideload). **Lose the keystore → you can't ship updates as the "same app" again, only as a brand new one.**
From a terminal:

bash
```
keytool -genkey -v \
  -keystore ~/handy-release.keystore \
  -alias handy \
  -keyalg RSA -keysize 2048 \
  -validity 10000 \
  -storepass "<long-random-password>" \
  -keypass "<long-random-password>" \
  -dname "CN=Satvik Bansal, OU=Handy, O=Handy, L=Bangalore,
ST=Karnataka, C=IN"
```
Then **back up** handy-release.keystore **plus both passwords** to a password manager (1Password / Bitwarden) AND a cold copy (USB drive, encrypted email-to-self). If you only remember one piece of advice from this whole answer, make it this one — Play App Signing later mitigates the "lost upload key" risk but you still need the upload key to push the very first build.
Step 2 — Wire signing into the Gradle build
Two parts: a keystore.properties file (gitignored, holds the passwords) and a signingConfigs block.
Create keystore.properties at the repo root (and .gitignore it):

properties
```
storeFile=/Users/satvik.bansal/handy-release.keystore
storePassword=<the password you used above>
```

```
keyAlias=handy
keyPassword=<the password you used above>
```
Add to app/build.gradle.kts, near the top of the android { ... } block:

kotlin
```
val keystoreProps = java×util×Properties()×apply {
    val f = rootProject.file("keystore.properties")
    if (f.exists()) load(f.inputStream())
}

android {
    // ... existing config ...

    signingConfigs {
        create("release") {
            keyAlias = keystoreProps×getProperty("keyAlias")
            keyPassword =
keystoreProps×getProperty("keyPassword")
            storeFile =
keystoreProps×getProperty("storeFile")?.let { file(it) }
            storePassword =
keystoreProps×getProperty("storePassword")
            // V2 + V3 + V4 signature schemes — required for
Android 11+
            enableV2Signing = true
            enableV3Signing = true
            enableV4Signing = true
        }
    }

    buildTypes {
        debug { /* unchanged */ }
        release {
            // ... existing minify/shrink/proguard ...
            signingConfig =
signingConfigs.getByName("release")
        }
    }
}
```
Add keystore.properties and *.keystore to .gitignore.

Step 3 — Build the release APK

bash
```
./gradlew :app:assembleRelease
```
Output: app/build/outputs/apk/release/app-release.apk. Verify it's signed:

bash

```
$ANDROID_HOME/build-tools/<version>/apksigner verify --
verbose app/build/outputs/apk/release/app-release.apk
```
Should report Verified using v2 scheme: true and v3 scheme: true.

Step 4 — Smoke test the release APK locally

Release builds with isMinifyEnabled = true can crash where debug builds don't, because R8 strips classes that the app reaches only via reflection (Hilt, Compose Preview helpers, Kotlin serialization, etc.). Before sharing, install on at least one device:

bash
```
adb install -r app/build/outputs/apk/release/app-release.apk
```
Run through the core flows: long-press voice → response → TTS, type a query, trigger a tap-for-me, open Photos and confirm "In Photos…" greeting, swipe to YouTube and watch the panel follow. If anything crashes, fix the R8 rules in app/proguard-rules.pro (usually -keep class com.handy.** { *; } is too broad; keep just the reflection-touched classes).

Step 5 — Upload to Drive and share

Drag app-release.apk into a Google Drive folder. Right-click → Share → **Anyone with the link → Viewer**. Copy the link. Send to testers with a 3-line instruction set:

1. Open the link on your Android phone.
2. Tap the APK to download. Chrome/Drive will warn "This type of file can harm your device" — tap OK.
3. The first time, Android will ask "Allow Drive/Chrome to install unknown apps?" — tap Settings → toggle on for that app, then return and tap Install.
4. After install, Google Play Protect will show a one-time scan warning. Tap Install anyway. We've enrolled Handy with Play Protect's review queue; this banner will diminish over the first few days.

Step 6 — Bake in feedback collection BEFORE you send the link

For sideload distribution there's no automatic crash console. Two options, pick one:

- **Free, simplest**: bundle Firebase Crashlytics. Add the Crashlytics Gradle plugin + a google-services.json in app/. Crashes auto-upload, you see them in the Firebase console. Free tier handles unlimited crashes.
- **Privacy-first alternative**: send a one-shot Slack/Telegram webhook from Thread.setDefaultUncaughtExceptionHandler with a redacted stack trace. About 30 lines of Kotlin, no SDK dependency.

Also add a one-line "Send feedback" entry in Settings that opens mailto: to your address with the version code/name pre-filled in the body. Catches the 90% of feedback that isn't a crash.

Step 7 — Know the limits of sideload distribution in 2025–2026

A few things to be aware of so you size expectations right:

- **Play Protect friction**: every install will show a one-time "Sent for review" or "Unknown app" banner. Most testers tap through it; non-

technical users get scared off.

- **Android 13+ "Restricted Settings"**: on a fresh sideloaded install, Android disables access to AccessibilityService and a few other sensitive APIs until the user goes to Settings → tap the app → "⋮" → "Allow restricted settings". You need to document this for testers — without it, Handy's tap-for-me silently fails to enable. Add a screen in your tester onboarding.
- **Google's 2026 developer verification**: starting **September 2026** in Brazil, Indonesia, Singapore, Thailand, and rolling out globally through 2027, Google will require developer identity registration even for sideloaded apps on Google-certified devices. You'll need to register your signing keys with Google. There's a free "limited distribution account" for hobbyists that caps at 20 devices and doesn't require government ID; a fuller registration unlocks unlimited sideload. India is not in the first wave, so you have runway, but this is the wall the APK-only strategy hits.

Bottom line: APK via Drive is great for 2–20 closed friends/colleagues over the next few weeks. Beyond that, the Play closed-test track is both easier and more durable.

Part 3 — Path B: Play Store V1 release

Once you've collected sideload feedback and fixed the obvious bugs, here's the Play Store path.

Step 1 — Pre-flight: tighten these in the app first

Item	Status	Action
Signing config	❌ needs setup	Done in Path A above — same keystore.
Privacy policy	⚠️ in repo only	Host the PRIVACY_POLICY.md at a public URL. Cheapest: GitHub Pages on a handy-privacy repo, free. Net cost: $0.
App icon	⚠️ verify	Play requires a 512 × 512 PNG icon (high-res) plus a 1024 × 500 feature graphic.

Screenshots	❌ need taking	Minimum 2, max 8 per device size (phone, 7" tablet, 10" tablet). Take 4–6 phone screenshots.
Short description	❌ need writing	80-char punch line. Currently nothing official.
Full description	❌ need writing	4,000 chars. The capability-truth manifest from CAPABILITIES.yaml is your honest copy.
Data safety form	❌ need filling	Declare every data type sent to Sarvam, Claude, Brave, Jina. Play Console UI walks you through it.
AccessibilityService disclosure	✅ already in onboarding	Per the onboarding redesign. Make sure the disclosure copy matches the Play declaration exactly.
Foreground service types	✅ already declared	specialUse, microphone, mediaProjection.
Target audience	❌ declare	"13+" or "18+" depending on what you want; AccessibilityService apps with adult-facing content tend to declare 18+ to avoid extra scrutiny.

Step 2 — Create the Play Console account ($25 one-time)
Go to play.google.com/console. Sign in with the Google account you want to own the developer identity (this can't be transferred easily, so pick deliberately — many developers create a dedicated account rather than use their personal Gmail).
You'll be asked to choose:

- **Personal account** — fastest, free identity verification, but triggers the **12-tester / 14-day closed-test requirement** before you can ship to production. ID verification (driver's license / passport / Aadhaar)

required.
- **Organization account** — requires a D-U-N-S number (free, takes 5–7 business days from Dun & Bradstreet), and proof of business existence. Skips the 12/14 requirement. Worth it only if you're shipping under a registered entity.

For Handy as a one-person project: **personal account is the right choice.** You're going to do closed testing anyway, the 12/14 isn't really a constraint if you've already done sideload testing.
Pay the $25 fee with a credit/debit card (no prepaid, no virtual). Done once, never again.
ID verification takes 24–72 hours typically. You can configure most of your app while it's pending; you just can't publish.
Step 3 — Build an Android App Bundle (.aab), not an APK
Play Store requires Android App Bundle (.aab) format since August 2021. From the Bundle, Google generates device-specific APKs.

bash
```
./gradlew :app:bundleRelease
```
Output: app/build/outputs/bundle/release/app-release.aab. This is what you upload.
Step 4 — Enroll in Play App Signing
When you upload your first .aab, Play asks "do you want Google to manage your signing key?" — **say yes**. This means:

- You sign your .aab with your **upload key** (the keystore from Path A).
- Google re-signs the distributed APKs with the **app signing key** they store securely.
- If you lose your upload key, you can request a reset from Google (you couldn't if you'd opted out).
- Required for any new app on Play since August 2021 anyway.

Step 5 — Set up the Closed Testing track
Closed testing is a private track where you list specific testers by email or by Google Group. This is what unlocks the 14-day timer.
In Play Console: **Testing → Closed testing → Create track**. Upload your .aab. Add testers — minimum 12 opt-ins for the personal-account requirement.
Important rules Google enforces algorithmically as of 2026:

- **Real devices only** — Google's 2026 algorithm rejects testers who install only on emulators. Test accounts that have only installed your app and nothing else also raise flags.
- **Continuous 14 days** — the clock starts when a tester opts in. If a tester opts out and back in mid-period, their clock resets.
- **Active engagement** — Google looks for app opens, not just installs. Tell testers explicitly: "open the app at least once every few days for the next two weeks."
- **Multiple device types help** — the algorithm prefers a mix of Pixel + Samsung + others over 12 Pixels.

After 14 days with 12 engaged testers, Play Console unlocks the "Apply for

production access" button.
For Handy, your 12 testers are people you can text. Friends with Android phones, colleagues, family. They install via a Play link you give them — much friendlier than the sideload flow because there's no Play Protect warning.

Step 6 — Fill out the app content declarations

In Play Console → **App content**, you must complete:

- **Privacy policy URL** — paste your GitHub Pages link.
- **App access** — "All functionality is available without restrictions" or "All or some functionality is restricted." Handy needs an API key, so describe how a reviewer can test (probably provide a Claude API key for the review, or implement a "demo mode" that lets the reviewer poke around without keys).
- **Ads** — no.
- **Content rating** — fill in IARC questionnaire. Handy doesn't have user-generated content, ads, gambling, or violence; should land at "Everyone" or "Teen."
- **Target audience** — pick 13+ or 18+. AccessibilityService apps with general-purpose use usually pick 18+.
- **News app** — no.
- **COVID-19 contact tracing** — no.
    - **Data safety** — the big one. Walk through each data type the app collects or transmits:
    - **Audio recordings** — collected (microphone), transmitted (to Sarvam if enabled, to system STT otherwise), not stored.
    - **Voice / sound recordings** — same.
    - **Photos** — none (Handy reads screen text, not photos).
    - **Other text user content** — collected (typed messages), transmitted (to Claude / Brave / Jina / Sarvam), stored locally for history, can be cleared.
    - **App activity** — interactions with apps via Accessibility, used for app functionality, not transmitted as such (the screen text is summarized into the LLM prompt).
    - **Crash logs / Diagnostics** — collected if you enabled Crashlytics.
    - **Device or other IDs** — collected (advertising ID? probably not; Firebase Installation ID if you used Crashlytics).
- Be honest. Misdeclaration is a fast track to suspension.

Step 7 — The AccessibilityService declaration — this is Handy's risk moment

In Play Console → **App content → Sensitive permissions and APIs**. AccessibilityService is one of the most-rejected categories.

The key Google rule: **only apps that primarily help users with disabilities can set** android:isAccessibilityTool="true". Handy is a productivity / agent app, not a disability tool. So isAccessibilityTool **must be FALSE** in your accessibility-service config XML. Verify this.

Because you're NOT claiming the accessibility-tool exemption, you must:

1. Complete the **Accessibility Declaration** form in Play Console. Walks through what you use the API for. Be specific: "Reading visible UI text to ground LLM responses, identifying tap targets the user explicitly asks the assistant to point to or tap on their behalf after explicit confirmation."
2. Demonstrate the **prominent disclosure** in-app: a screen shown before AccessibilityService is enabled that explains what the app will do with it, in plain language, with an explicit accept/decline. Handy's ActionDisclosureActivity is this screen. Make sure the copy matches the Play declaration.
3. Provide a **video** in the Play Console submission showing the disclosure flow + an example end-to-end use. 30–60 second screen recording. The video is the single highest-leverage thing you can submit — reviewers watch it.
4. Be ready to provide a **video specifically demonstrating that LLM-suggested actions REQUIRE user confirmation before executing**. Google's updated 2026 policy explicitly bans apps that "autonomously initiate, plan, and execute actions" via AccessibilityService. Your tap-for-me confirmation sheet is exactly the proof point. Show it on video.

What rejection looks like and how to handle it: Play will write a generic "your app does not meet our accessibility policy" message. Don't argue the policy in the appeal. Instead, write back with specific in-app evidence: timestamped screen recording showing each action gate, list the policy bullets you comply with, link to your privacy policy. Reviewers approve 60–70% of accessibility apps on first appeal if the appeal is concrete.

Step 8 — Submit & wait

After closed testing completes (14 days + 12 testers), in Play Console:

1. **Testing → Closed testing → Promote release → Production** (or create a new Production release from scratch — same .aab).
2. Fill in release notes.
3. Submit for review.

Review timeline expectations:

- New apps in 2026: typically **3–7 days** for first review.
- Sensitive permission apps (AccessibilityService, etc.): often **7–14 days**, sometimes longer.
- Subsequent updates: **a few hours to 2 days**, unless they touch sensitive permissions.

You'll get one of three outcomes: Approved, Rejected with reason, or "We need more info" with a specific question. Iterate.

Step 9 — After launch

- Monitor the **Crashes & ANRs** dashboard in Play Console. You don't need Crashlytics on Play; Play's native crash collection is excellent.
- Watch **Vitals → Bad behavior** for ANRs and excessive wake-locks.
- Watch user reviews. A 4.0+ rating is the cutoff below which Play's algorithmic surfacing drops sharply.

- Plan version updates: each release needs a versionCode bump (you're at 1 — bump to 2 for the next push). versionName is for humans (0.1.1, 0.2.0, etc.).

Part 4 — Costs & time, all in

Item	Cost	Time
Keystore creation	$0	5 min
Signing config + first APK	$0	30 min
Google Drive APK distribution	$0	10 min/release
Privacy policy hosting (GitHub Pages)	$0	30 min
Firebase Crashlytics (optional)	$0	1 hour
Play Console account fee	**$25 one-time**	account opens immediately, ID verification 24–72h
Closed testing (12 testers / 14 days)	$0	**14+ days wait**
Production review	$0	3–14 days
Updates after that	$0	hours per submission

Realistic timeline for Handy:

- This week: keystore + signed APK + Drive distribution to your first 5–10 testers.
- Weeks 1–2 of sideload: collect feedback, fix obvious bugs, iterate on APK.
- Week 2: create Play Console account, fill in store listing, upload first .aab to closed test, invite 12+ real testers.
- Weeks 3–4: closed-test 14-day clock runs. Use this time to write the AccessibilityService disclosure video and the data-safety form.
- Week 4 or 5: promote to production, submit for review.
- Week 5 or 6: live on Play Store, assuming clean review or one round of clarifications.

Total from sitting down today to Play Store live: 5–6 weeks for a clean run, 8–10 weeks if you hit one AccessibilityService re-review.

The keystore + signed APK piece is what I'd do tonight. Everything else can be staged on top.

Sources:

- Google Play Console developer account fee 2026 — $25 one-time, no annual
- App testing requirements for new personal developer accounts — official Play Console help

- Google Play Closed Testing Policy 2026 — 12 testers / 14 days explained
- Use of the AccessibilityService API — Play Console policy
- Permissions and APIs that Access Sensitive Information — Play Console help
- Best practices for prominent disclosure and consent — Play Console help
- Android 14/15 sideloading restrictions and "Install unknown apps" flow
- Android 15 Restricted Settings / Enhanced Confirmation Mode for sideloaded apps
- Google's 2026 developer verification for sideloaded apps — rollout timeline
- Distribute Android apps without Google Play — 4 methods comparison, 2026

okay step 1,2,3 are confusing. can you be more detailed with steps please? i want to close this today

Got it — let me walk through Steps 1–3 with every keystroke so you can close this today. I'll assume macOS terminal and no prior keytool experience. Let me check your repo state first so my instructions match what's actually there.
Searched 3 patterns, read a file

Searched 3 patterns, read a file

Confirmed — .gitignore already ignores *.keystore (good),
but keystore.properties needs adding. No existing keystore. Here's the complete walkthrough.

Step 0 — Confirm keytool works (1 minute)

keytool is part of the Java JDK. Open Terminal (⌘+Space → "Terminal" → Enter) and run:

bash
```
keytool -help
```
Case A — you see a wall of help text starting with "Key and Certificate Management Tool": ✅ skip to Step 1.
Case B — you see "No Java runtime present, requesting install" or "command not found": you need to point Terminal at Android Studio's bundled JDK. Run:

bash
```
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
export PATH="$JAVA_HOME/bin:$PATH"
```

```
keytool -help
```
Should print help text now. Note: this export only lasts for this Terminal window. If you close it and reopen, run the two export lines again at the start of the new session. (To make it permanent later: echo 'export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"' >> ~/.zshrc.)

Step 1 — Create the release keystore (5 minutes)
1.1 — Pick the file location. I recommend ~/handy-release.keystore — that's /Users/satvik.bansal/handy-release.keystore. This is **outside** your repo, in your home directory. That's intentional — the keystore should never accidentally land in git.
1.2 — Pick two passwords. Same password works for both (store password + key password). Make it long and random and store it in a password manager NOW before you forget it. Example: open 1Password / Bitwarden / Apple Keychain, create a new entry called "Handy Android keystore", generate a 24-char password, save it. Keep that entry open in another window.
1.3 — Run the keytool command. Copy this exact block into Terminal. Replace the two <paste-your-password-here> strings with the password you just saved.

bash
```
keytool -genkey -v \
  -keystore ~/handy-release.keystore \
  -alias handy \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -storepass "<paste-your-password-here>" \
  -keypass "<paste-your-password-here>" \
  -dname "CN=Satvik Bansal, OU=Handy, O=Handy, L=Bangalore, ST=Karnataka, C=IN"
```
Hit Enter.
1.4 — Expected output. You'll see something like:

```
Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 10,000 days
    for: CN=Satvik Bansal, OU=Handy, O=Handy, L=Bangalore, ST=Karnataka, C=IN
[Storing /Users/satvik.bansal/handy-release.keystore]
```
If you see a keytool warning about "PKCS12 is an industry standard format" — that's fine, ignore it. The keystore is valid.
1.5 — Verify the file exists.

bash
```
ls -lh ~/handy-release.keystore
```
You should see something like -rw------- 1 satvik.bansal staff 2.6K ... handy-release.keystore. If the file is there, you're good.
1.6 — Verify the key is readable.

bash
```
keytool -list -keystore ~/handy-release.keystore -storepass
"<paste-your-password-here>"
```
You should see:

```
Keystore type: PKCS12
Keystore provider: SUN
Your keystore contains 1 entry
handy, ..., PrivateKeyEntry,
Certificate fingerprint (SHA-256): AB:CD:EF:...
```
The handy line confirms the alias matches what we'll reference in Gradle. ✅
1.7 — Back it up RIGHT NOW. Drop a copy in iCloud Drive, a USB drive, or upload encrypted to your password manager. If you lose this file plus the passwords, you can never update your app again under the same identity. Do not skip this.

bash
```
# Optional convenience: copy to iCloud Drive
cp ~/handy-release.keystore ~/Library/Mobile\ Documents/
com~apple~CloudDocs/handy-release.keystore.backup
```

Step 2 — Wire signing into Gradle (10 minutes)
Three sub-steps: create keystore.properties, update .gitignore, edit app/build.gradle.kts.
2.1 — Create keystore.properties **at the repo root.** Open Terminal, navigate to your repo:

bash
```
cd /Users/satvik.bansal/Desktop/Handy.android/HandyV2
```
Create the file using your editor of choice. Easiest: open it directly in your editor (VS Code / Android Studio / TextEdit). Or use this Terminal one-liner — replace the password placeholder:

bash
```
cat > keystore.properties <<EOF
storeFile=/Users/satvik.bansal/handy-release.keystore
storePassword=<paste-your-password-here>
keyAlias=handy
keyPassword=<paste-your-password-here>
EOF
```
The file should now exist at /Users/satvik.bansal/Desktop/Handy.android/HandyV2/keystore.properties. Verify:

bash
```
cat keystore.properties
```

You should see all four lines printed back. ✅
2.2 — Add keystore.properties **to** .gitignore**.** Your .gitignore already excludes *.keystore (good — the keystore file itself won't accidentally commit even if you copied it into the repo). But keystore.properties isn't ignored yet. Run:

bash
```
echo "" >> .gitignore
echo "# Signing config — contains keystore passwords"
>> .gitignore
echo "keystore.properties" >> .gitignore
```
Verify:

bash
```
tail -3 .gitignore
```
Should show:

```
# Signing config — contains keystore passwords
keystore.properties
```
✅ Now even if you git add . by accident, this file stays out.
2.3 — Edit app/build.gradle.kts**.** Open the file in your editor: /Users/satvik.bansal/Desktop/Handy.android/HandyV2/app/build.gradle.kts.
You need to make **three** small changes inside the android { ... } block.
Change 1 of 3 — Add a properties loader at the very top of the file, AFTER the existing plugins { ... } block and BEFORE android { ... } starts.
Find this section (lines 1–9):

kotlin
```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.serialization)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt)
}

android {
```
Insert this block between the closing } of plugins and the opening of android {:

kotlin
```
val keystoreProps = java.util.Properties().apply {
    val f = rootProject.file("keystore.properties")
    if (f.exists()) load(f.inputStream())
}
```
So lines 1–13 should now look like:

kotlin

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.serialization)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt)
}

val keystoreProps = java×util×Properties()×apply {
    val f = rootProject.file("keystore.properties")
    if (f.exists()) load(f.inputStream())
}

android {
```

Change 2 of 3 — Add a signingConfigs block INSIDE android { … }, BEFORE the buildTypes { … } block.
Find this section (around line 25 in the original file):

kotlin

```
    buildTypes {
        debug {
            isDebuggable = true
            isMinifyEnabled = false
        }
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-
optimize.txt"),
                "proguard-rules.pro",
            )
        }
    }
```

Insert this block ABOVE the buildTypes { … } block:

kotlin

```
    signingConfigs {
        create("release") {
            val storeFilePath =
keystoreProps×getProperty("storeFile")
            if (storeFilePath != null) {
                storeFile = file(storeFilePath)
                storePassword =
```

```
keystoreProps×getProperty("storePassword")
                keyAlias =
keystoreProps×getProperty("keyAlias")
                keyPassword =
keystoreProps×getProperty("keyPassword")
            }
        }
    }

    buildTypes {
        debug {
            isDebuggable = true
            isMinifyEnabled = false
        }
        release {
            ...
        }
    }
```

(That if (storeFilePath != null) guard means: if a teammate clones the repo without keystore.properties, the build doesn't crash — they just can't produce a signed release. Debug builds still work fine.)

Change 3 of 3 — Inside the existing release { … } block, add a single line that references the signing config. Find this:

kotlin

```
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-
optimize.txt"),
                "proguard-rules.pro",
            )
        }
```

Change it to:

kotlin

```
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            signingConfig =
signingConfigs.getByName("release")
            proguardFiles(
                getDefaultProguardFile("proguard-android-
optimize.txt"),
                "proguard-rules.pro",
            )
```

```
        }
```
Save the file.
2.4 — Sanity check the file compiles. Back in Terminal:

bash
```
cd /Users/satvik.bansal/Desktop/Handy.android/HandyV2
./gradlew :app:tasks --quiet | head -5
```
If you see a list of Gradle tasks (lines starting with "Android tasks", "Build tasks", etc.) it parsed correctly. If you see a red error like "Unresolved reference: keystoreProps" or "Expecting an element", the edit didn't land cleanly — re-check the indentation and placement.

Step 3 — Build the signed APK (3–5 minutes, mostly waiting)
3.1 — Make sure you're in the repo root.

bash
```
cd /Users/satvik.bansal/Desktop/Handy.android/HandyV2
pwd
```
Should print /Users/satvik.bansal/Desktop/Handy.android/HandyV2. ✅
3.2 — Make gradlew **executable** (one-time, harmless if already):

bash
```
chmod +x gradlew
```
3.3 — Run the release build.

bash
```
./gradlew :app:assembleRelease
```
Hit Enter. On the first run this takes 3–8 minutes because Gradle downloads R8/ProGuard tooling. Watch for:

- A bunch of > Task :core:compileKotlin, :android-runtime:compileKotlin, :app:compileReleaseKotlin — normal.
- > Task :app:minifyReleaseWithR8 — R8 is shrinking the code. This is where things can go wrong (see "if it fails" below).
- > Task :app:packageRelease — bundling.
- Final line should be:

```
  BUILD SUCCESSFUL in 4m 12s
```
If you see BUILD SUCCESSFUL, you're done with the hard part. 🎉
3.4 — Locate the APK.

bash
```
ls -lh app/build/outputs/apk/release/
```
You should see app-release.apk (probably 15–40 MB). That's your shippable file.
3.5 — Verify it's actually signed. Find your Android SDK's apksigner:

bash
```
ls ~/Library/Android/sdk/build-tools/
```
Pick the highest version number (e.g. 35.0.0), then:

bash
```
~/Library/Android/sdk/build-tools/35.0.0/apksigner verify --verbose app/build/outputs/apk/release/app-release.apk
```
You should see:

```
Verifies
Verified using v1 scheme (JAR signing): false
Verified using v2 scheme (APK Signature Scheme v2): true
Verified using v3 scheme (APK Signature Scheme v3): true
Number of signers: 1
```
Verified using v2 scheme: true is the line you care about. ✅ Your APK is ready to share.

3.6 — Optional but recommended: install on your own device first.

bash
```
adb install -r app/build/outputs/apk/release/app-release.apk
```
If adb isn't found, it's at ~/Library/Android/sdk/platform-tools/adb. Connect your phone via USB, enable USB debugging in Developer Options, and run the install. Walk through the core Handy flows for 10 minutes — long-press voice, tap-for-me, app switch with the panel open. Release builds occasionally crash where debug builds don't (R8 stripping); better to catch it now than after you've shared the link.

If something fails

Error	Cause	Fix
keytool: command not found	JAVA_HOME not set	Re-run the export from Step 0 in the same Terminal window.
Gradle error: Unresolved reference: keystoreProps	The properties loader block landed inside android { } instead of above it	Move it to ABOVE android { (between plugins { ... } and android {).

Gradle error: storeFile must not be null	keystore.properties not at repo root, or path inside it is wrong	Verify cat keystore.properties from repo root prints all 4 lines, and storeFile=/Users/satvik.bansal/handy-release.keystore matches the real path.
Gradle error: Keystore was tampered with, or password was incorrect	Password in keystore.properties doesn't match what you typed in Step 1.3	Recheck the password. If you forgot it, you'll have to regenerate the keystore (Step 1) and update keystore.properties.
minifyReleaseWithR8 FAILED — ... missing rule	R8 stripped a class the app reaches by reflection (Hilt / serialization)	Comment out isMinifyEnabled = true temporarily in release { }, rebuild, ship that. We can fix ProGuard rules separately.
Execution failed for task ':app:packageRelease' ... signature did not verify	Mid-edit signingConfig line typo	Re-check Change 3 of 3 — must read signingConfig = signingConfigs.getByName("release") exactly.
APK builds but app crashes on launch on the phone	R8 over-stripping	Same fix as the R8 row above — disable minify, rebuild, ship. We hunt the offending rule later.

When all three steps are done

You'll have:

- /Users/satvik.bansal/handy-release.keystore — your signing key. **Backed up in your password manager.**
- /Users/satvik.bansal/Desktop/Handy.android/HandyV2/keystore.properties — the password file. **Gitignored.**
- /Users/satvik.bansal/Desktop/Handy.android/HandyV2/app/build/outputs/apk/release/app-release.apk — your shareable file. **Drop in Drive.**

From here, Steps 4–5 (smoke test + Drive share) are 15 minutes. You'll be sending the link to testers tonight.

Ping me once you hit BUILD SUCCESSFUL or if anything throws an error — I can

debug it in real time.